Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Санкт-Петербургский политехнический университет Петра Великого»
—
Институт компьютерных наук и кибербезопасности
Высшая школа кибербезопасности

ЛАБОРАТОРНАЯ РАБОТА № 2

«ОРГАНИЗАЦИЯ ЦИФРОВОГО ВВОДА-ВЫВОДА»

по дисциплине «Аппаратные средства вычислительной техники»

Выполнил
студент гр. 5151003/40001 Корпусов Д.Н

<подпись>

Руководитель Макаров А.С

<подпись>

Санкт-Петербург
2025

Содержание

Цель работы 3
Ход работы 3-15
1. Теоретическая часть 3-5
1.1 Порты ввода-вывода МК 3
1.2 Подключение внешних устройств 3-4
1.3 Подсистема прерываний 4
1.4 Внешние прерывания INT0 и INT1 4
1.5 Энергонезависимая память EEPROM 4-5
1.6 Представление чисел со знаком 5
2. Практическая часть 6-15
2.1 Постановка задачи 6
2.2 Используемые порты и их назначение 6
2.3 Инициализация МК 7
2.4 Организация режимов работы гирлянды 7-8
2.5 Формирование частоты переключения состояний 8
2.6 Переключение режимов с помощью внешних прерываний 8-9
2.7 Индикация режима и состояния на PORTD 9
2.8 Ввод числа y через PORTC 9-10
2.9 Сохранение режима в EEPROM 10
2.10 Итог работы программы 11-15
Ответы на контрольные вопросы 16-17
Вывод 18
Листинг «.asm-файл» 19-29

ЦЕЛЬ РАБОТЫ

Изучение основ работы с цифровыми портами ввода-вывода микроконтроллера ATmega32. Получение практических навыков по обработке внешних прерываний и организации ввода-вывода с помощью механизма прерываний.

ХОД РАБОТЫ

1. Теоретическая часть

1.1 Порты ввода-вывода МК

Связь микроконтроллера с внешними устройствами осуществляется через порты ввода-вывода, каждый из которых представляет собой набор линий, соответствующих выводам микросхемы. Управление портами осуществляется с помощью трёх регистров: DDRx, PORTx и PINx.

Регистр DDRx определяет направление работы каждого вывода: при значении бита 1 линия работает как выход, при 0 — как вход. Регистр PORTx используется для записи логического уровня на выход или включения подтягивающего резистора во входном режиме. Регистр PINx применяется для считывания текущего состояния входа.

Таким образом, каждый вывод микроконтроллера может независимо работать как вход или как выход, что позволяет гибко организовывать взаимодействие с внешними устройствами.

1.2 Подключение внешних устройств

Для исследования работы портов ввода-вывода используются светодиоды и кнопки. Светодиоды подключаются к выводам микроконтроллера через ограничивающие резисторы и управляются записью логических уровней в регистр PORTx.

В разработанной схеме кнопки подключены таким образом, что при нажатии на входе микроконтроллера формируется логическая единица, а при отпускании — логический ноль.

Таким образом, активное состояние кнопки соответствует логической 1, что необходимо учитывать при программной обработке входных сигналов и настройке прерываний.

1.3 Подсистема прерываний

Подсистема прерываний позволяет микроконтроллеру реагировать на внешние события без постоянного опроса входов. При возникновении прерывания выполнение основной программы приостанавливается, и управление передаётся обработчику прерывания, расположенному по соответствующему адресу в таблице векторов.

Для работы прерываний необходимо: разрешить конкретное прерывание, включить глобальное разрешение прерываний (бит I в регистре SREG), реализовать обработчик прерывания. После выполнения обработчика управление возвращается в основную программу.

1.4 Внешние прерывания INT0 и INT1

В микроконтроллере ATmega32 внешние прерывания INT0 и INT1 используются для обработки сигналов с внешних входов. Их настройка осуществляется с помощью регистров MCUCR, GICR и GIFR.

Регистр MCUCR определяет условие срабатывания прерывания. В данной работе прерывания настроены на срабатывание по фронту сигнала (переход 0→1), что соответствует используемой схеме подключения кнопок.

Регистр GICR используется для разрешения прерываний INT0 и INT1, а регистр GIFR — для сброса флагов прерываний.

1.5 Энергонезависимая память EEPROM

EEPROM представляет собой энергонезависимую память микроконтроллера, предназначенную для хранения данных, сохраняющихся после выключения питания. В данной работе EEPROM используется для хранения текущего режима работы устройства.

Доступ к EEPROM осуществляется через регистры EEAR, EEDR и EECR. Для чтения необходимо задать адрес и установить бит EERE, после чего данные считываются из регистра EEDR. Для записи необходимо

дождаться окончания предыдущей операции, затем записать адрес и данные и установить биты EEMWE и EEWE.

Использование EEPROM позволяет сохранять выбранный режим работы и восстанавливать его после перезапуска микроконтроллера.

1.6 Представление чисел со знаком

В микроконтроллерах числа со знаком могут представляться в различных кодах: прямом, обратном и дополнительном. Наиболее распространённым является дополнительный код, так как он упрощает выполнение арифметических операций.

В данной работе используется команда NEG, которая формирует дополнительный код числа, позволяя получить значение −y. Это используется в третьем режиме работы для отображения числа и его отрицательного значения.

2. Практическая часть

2.1 Постановка задачи

В соответствии с вариантом 5 требовалось реализовать программу для микроконтроллера ATmega32, в которой регистры PORTA и PORTB работают в режиме «новогодней гирлянды». Программа должна обеспечивать три режима работы: вывод значений 0xFF и 0x00, вывод значений 0xAA и 0x55, а также вывод значений y и -y, где отрицательное значение формируется в дополнительном коде. Смена состояний в каждом режиме должна выполняться с частотой 2 Гц. Кроме того, требовалось обеспечить переключение режимов с помощью внешних прерываний INT0 и INT1, отображение текущего режима и состояния на PORTD, ввод числа y через PORTC при нажатой кнопке PD7, а также сохранение номера режима во внутренней памяти EEPROM с последующим восстановлением после сброса.

2.2 Используемые порты и их назначение

В разработанной программе порты микроконтроллера были распределены по функциональным задачам. Порты PORTA и PORTB использовались как выходные и служили для отображения текущего шаблона гирлянды. На этих портах поочерёдно выводились пары значений, определяемые выбранным режимом и текущим состоянием.

Порт PORTD использовался одновременно и как входной, и как выходной. Разряды PD0 и PD1 применялись для индикации номера текущего режима, разряд PD4 использовался для отображения текущего состояния внутри режима. Разряды PD2 и PD3 работали как входы внешних прерываний INT0 и INT1, обеспечивая циклическое переключение режимов вперёд и назад соответственно. Разряд PD7 применялся как управляющий вход для перехода в режим ввода числа y.

Порт PORTC был настроен на ввод и использовался для считывания восьмиразрядного значения y, задаваемого с помощью кнопок.

2.3 Инициализация МК

В начале программы выполнялась инициализация стека, необходимая для корректной работы подпрограмм и обработчиков прерываний. Вершина стека устанавливалась в конец области оперативной памяти SRAM с помощью регистров SPH и SPL.

После настройки стека выполнялась инициализация портов ввода-вывода. Регистры DDRA и DDRB заполнялись значением 0xFF, переводя все линии этих портов в режим вывода. Для порта PORTD устанавливалось значение 0x13, что переводило линии PD0, PD1 и PD4 в режим вывода, а остальные разряды оставляло входами. Регистр DDRC обнулялся, в результате чего весь порт PORTC настраивался на ввод.

Далее из памяти EEPROM считывался сохранённый ранее номер режима. Если прочитанное значение выходило за пределы допустимого диапазона 1…3, то в качестве корректного начального значения устанавливался режим 1. После этого инициализировались внутренние переменные программы: STATE устанавливался в 0, а переменной y присваивалось начальное значение 0x55, предусмотренное условием задания.

Затем выполнялась начальная индикация текущего режима и состояния на PORTD, после чего настраивались внешние прерывания INT0 и INT1 и глобально разрешалась система прерываний командой SEI.

2.4 Организация режимов работы гирлянды

Основная логика программы строится на двух переменных: MODE и STATE. Переменная MODE определяет номер текущего режима работы, а STATE задаёт одно из двух состояний внутри выбранного режима.

В первом режиме на порты PORTA и PORTB поочерёдно выводятся значения 0xFF и 0x00, что соответствует включению и выключению всех светодиодов. Во втором режиме выводятся значения 0xAA и 0x55, формирующие чередующийся двоичный рисунок. В третьем режиме выводятся значение y, считанное с PORTC, и значение -y, вычисляемое в дополнительном коде с помощью команды NEG.

Выбор требуемого режима осуществляется в подпрограмме SHOW_MODE, где производится последовательное сравнение содержимого регистра r18 с числами 1, 2 и 3. После определения режима выполняется проверка переменной STATE, и в зависимости от её значения на порты PORTA и PORTB записывается один из двух шаблонов.

Таким образом, каждая комбинация MODE и STATE однозначно определяет выводимое значение на гирлянду.

2.5 Формирование частоты переключения состояний

Для обеспечения частоты переключения состояний 2 Гц использовалась программная задержка. Поскольку один полный цикл гирлянды состоит из двух состояний, длительность каждого состояния должна составлять примерно 250 мс.

Задержка была реализована через две подпрограммы: delay_31ms и DELAY_250MS. Подпрограмма delay_31ms формирует малый интервал задержки на основе вложенного программного цикла. Подпрограмма DELAY_250MS вызывает малую задержку восемь раз подряд, получая общее время, близкое к 250 мс.

После каждой задержки переменная STATE инвертируется с помощью операции EOR, что обеспечивает циклическое переключение между первым и вторым состоянием режима. В результате на портах PORTA и PORTB формируется мигающий сигнал с требуемой частотой.

2.6 Переключение режимов с помощью внешних прерываний

Смена режима работы реализована с использованием внешних прерываний INT0 и INT1. Для этого в таблице векторов по адресам INT0addr и INT1addr размещены переходы на обработчики EXT_INT0 и EXT_INT1.

В обработчике EXT_INT0 выполняется переход к следующему режиму. Значение MODE увеличивается на единицу, а при выходе за пределы диапазона 1…3 снова устанавливается равным 1. Таким образом реализуется циклический переход $1 \rightarrow 2 \rightarrow 3 \rightarrow 1$.

В обработчике EXT_INT1 реализован переход в обратном направлении. Если текущий режим равен 1, то устанавливается значение 3, иначе MODE уменьшается на единицу. Тем самым обеспечивается последовательность 1 ← 2 ← 3 ← 1.

После изменения режима в обоих обработчиках вызываются подпрограммы UPDATE_PORTD и SAVE_MODE, обеспечивающие обновление индикации и сохранение нового режима в EEPROM. Перед выходом из обработчиков восстанавливается регистр состояния SREG, после чего выполнение продолжается в основной программе командой RETI.

Использование механизма прерываний позволяет выполнять переключение режимов независимо от основной программы мигания гирлянды.

2.7 Индикация режима и состояния на PORTD

Для отображения служебной информации была реализована подпрограмма UPDATE_PORTD. Она формирует код текущего режима и состояния и выводит его на PORTD.

Перед записью новых данных подпрограмма считывает текущее значение регистра PORTD и очищает разряды PD0, PD1 и PD4. Затем в зависимости от значения MODE в младшие два разряда записывается код режима: 01 для режима 1, 10 для режима 2 и 11 для режима 3. После этого по значению STATE устанавливается или очищается бит PD4, отображающий текущее состояние внутри режима.

В результате на выходных линиях PORTD всегда присутствует актуальная информация о логическом состоянии программы, что облегчает визуальный контроль её работы.

2.8 Ввод числа y через PORTC

Ввод числа y реализован в подпрограмме READ_Y. Переход к ней осуществляется из основного цикла программы при активном сигнале на входе PD7. Пока кнопка PD7 удерживается нажатой, программа приостанавливает основной цикл работы гирлянды и многократно считывает

текущее состояние порта PINC в регистр r19, который используется как переменная y.

Как только управляющая кнопка отпускается, подпрограмма завершает работу и управление возвращается в основной цикл. С этого момента в третьем режиме отображается уже новое значение y и соответствующее ему значение -y.

Такой способ организации ввода соответствует условию задания, согласно которому ввод числа должен выполняться при нажатой кнопке PD7, а основной функционал на время ввода должен быть остановлен.

2.9 Сохранение режима в EEPROM

Для сохранения номера режима между перезапусками микроконтроллера использовалась внутренняя энергонезависимая память EEPROM. В программе был выбран адрес 0x00, по которому хранится значение MODE.

При запуске микроконтроллера выполняется чтение содержимого EEPROM по указанному адресу. Если считанное значение соответствует допустимому диапазону режимов, оно используется как текущий режим. В противном случае устанавливается режим 1 как безопасное начальное значение.

При каждом изменении режима в обработчиках прерываний вызывается подпрограмма SAVE_MODE. В ней сначала выполняется ожидание завершения предыдущей операции записи, затем в регистр адреса EEPROM заносится выбранный адрес, в регистр данных — текущее значение MODE, после чего через установку битов EEMWE и EEWE запускается операция записи.

Таким образом обеспечивается сохранение номера режима даже после аппаратного сброса или выключения питания.

2.10 Итог работы программы

В результате была разработана программа для микроконтроллера ATmega32, реализующая все основные требования варианта 5 лабораторной работы. Программа обеспечивает три режима работы гирлянды на портах PORTA и PORTB, переключение состояний с частотой 2 Гц, изменение режима по внешним прерываниям INT0 и INT1, индикацию режима и состояния на PORTD, ввод числа y через PORTC при нажатой кнопке PD7, а также сохранение текущего режима во внутренней памяти EEPROM.

В ходе выполнения работы были закреплены навыки настройки цифровых портов ввода-вывода, организации внешних прерываний, работы со стеком и регистром состояния, а также чтения и записи данных в энергонезависимую память микроконтроллера.

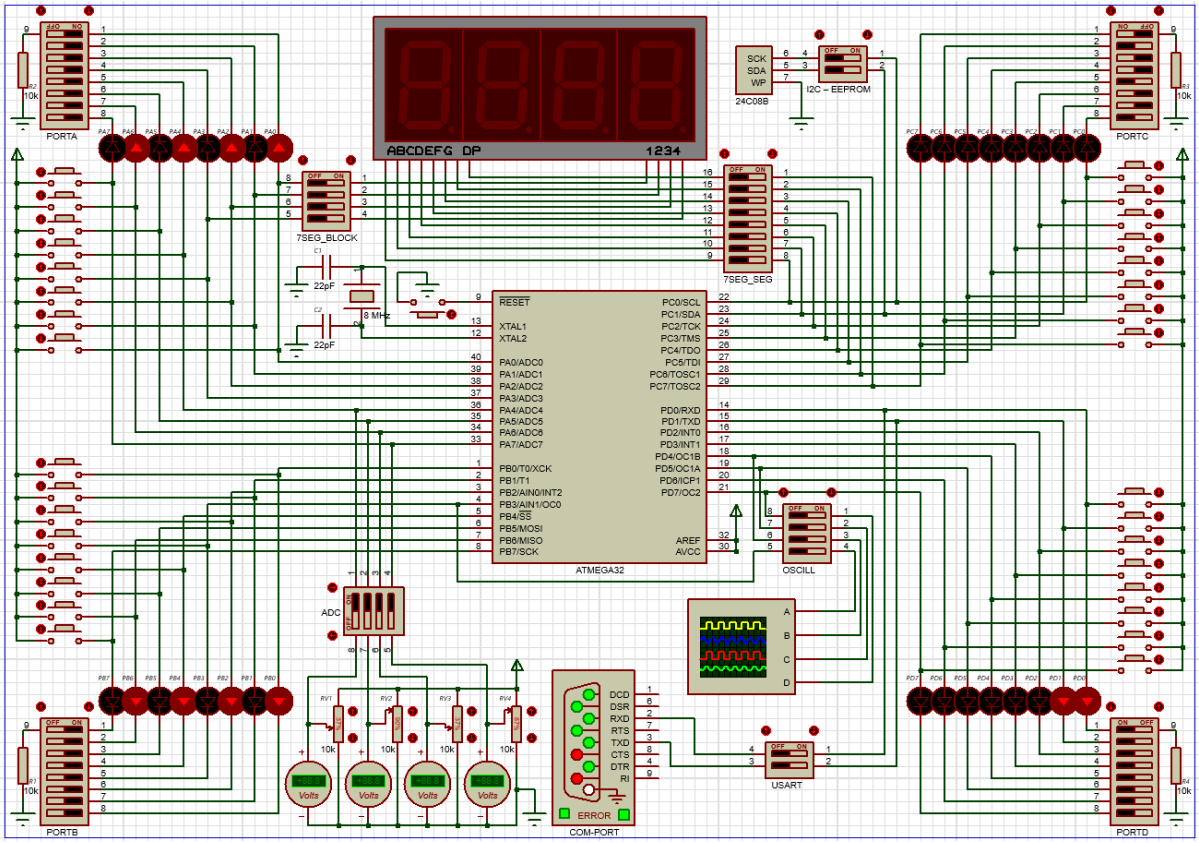


Рис. 1 «Схема установки»

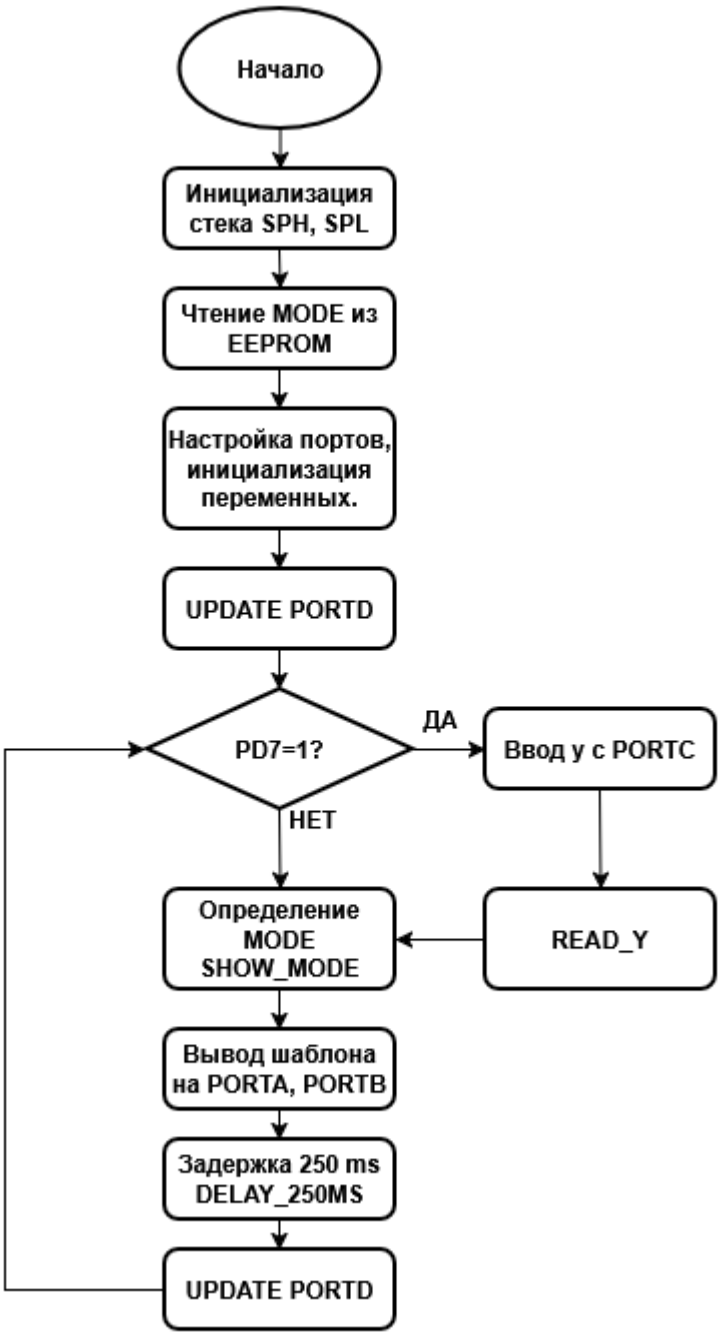


Рис. 2 «Блок-схема алгоритма работы программы»

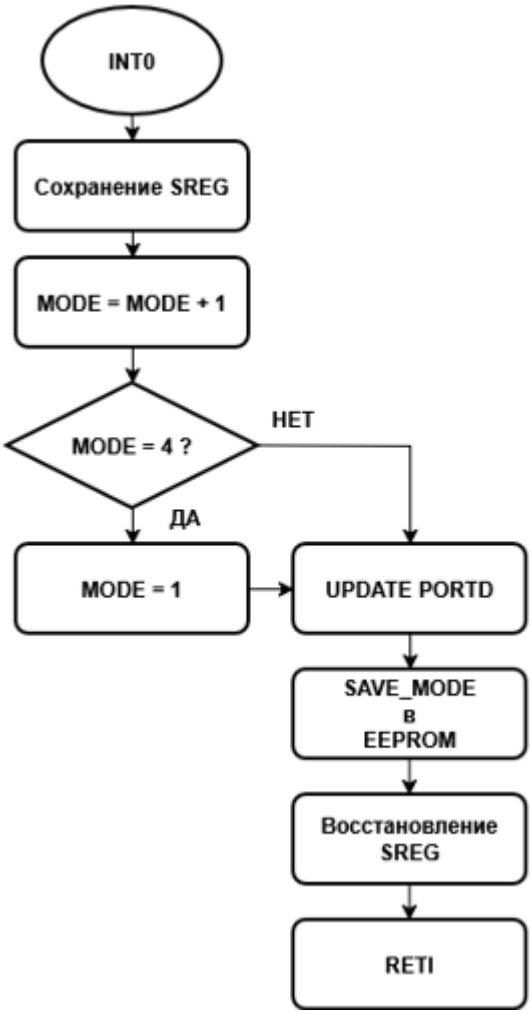


Рис. 3 «Блок-схема алгоритма прерывания INT0»

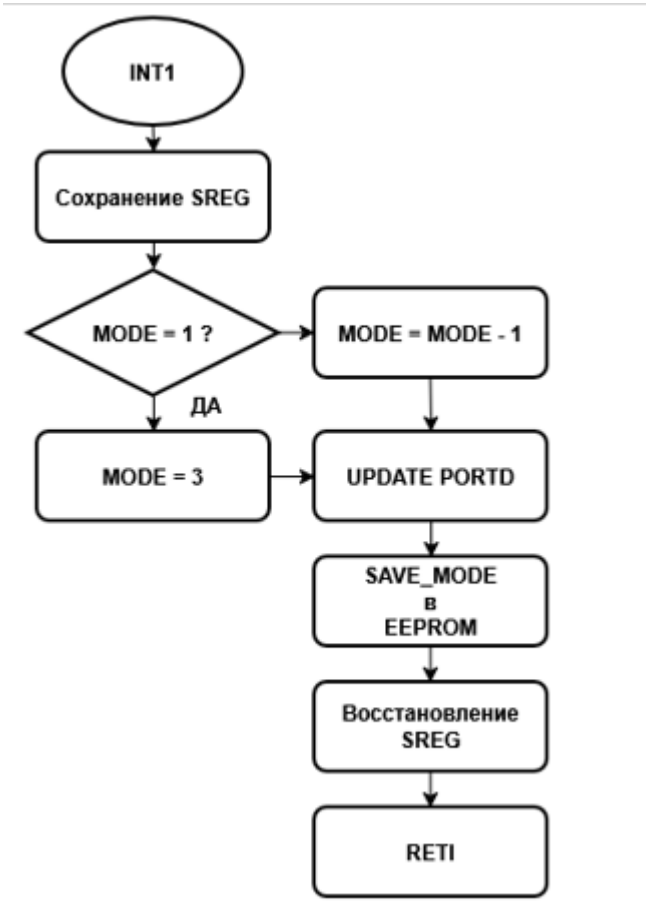


Рис. 4 «Блок-схема алгоритма прерывания INT1»

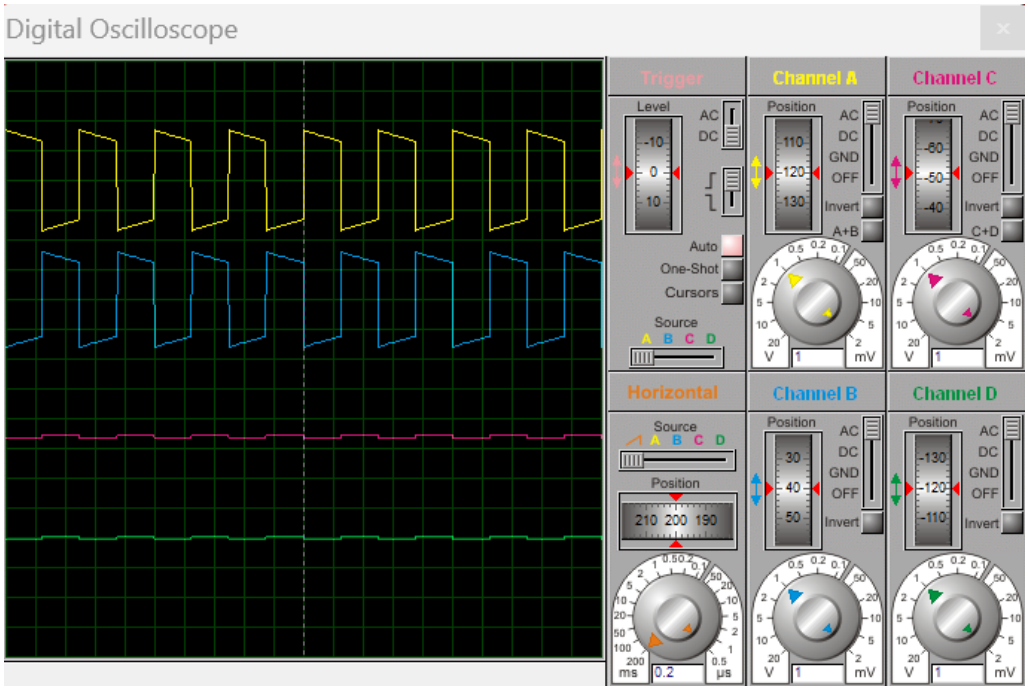


Рис. 5 «Временная диаграмма цифровых сигналов MODE=2»

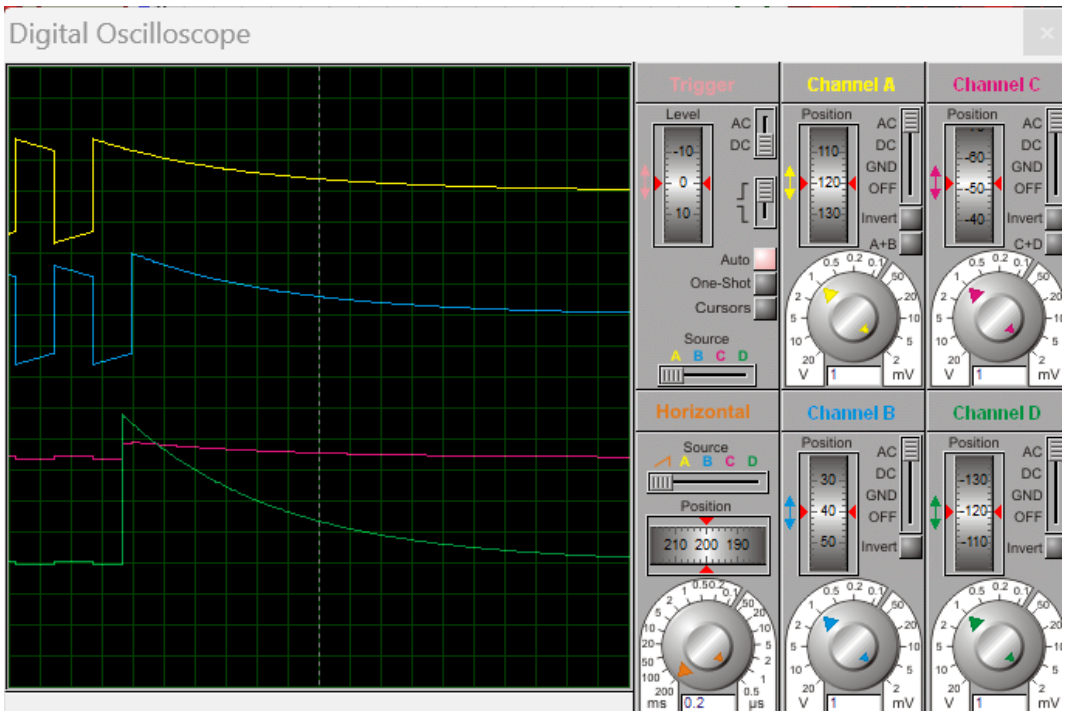


Рис. 6 «Временная диаграмма цифровых сигналов PD7=1»

На временной диаграмме представлены сигналы на портах микроконтроллера:

Жёлтый сигнал соответствует выходу PORTA/PORTB и отражает изменение состояния светодиодов.

Синий сигнал отображает значение переменной STATE (бит PD4), которое переключается с заданной частотой.

Зелёный сигнал отображает состояние входа PD7, при активации которого происходит ввод значения переменной y с порта PORTC. Временная диаграмма демонстрирует взаимосвязь между состоянием системы, режимами работы и управляющими воздействиями.

ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Какими способами можно подключить внешние устройства (светодиод, кнопку)?

Внешние устройства, такие как светодиоды и кнопки, подключаются к микроконтроллеру через порты ввода-вывода. Светодиоды подключаются через токоограничивающий резистор либо к источнику питания, либо к земле, в зависимости от выбранной схемы (активный высокий или низкий уровень). Кнопки подключаются с использованием подтягивающих резисторов: либо внутреннего pull-up резистора микроконтроллера, либо внешнего. В этом случае при нажатии кнопки на входе формируется устойчивый логический уровень (0 или 1), что обеспечивает корректное считывание сигнала.

2. Как реализуется подсистема прерываний в AVR?

Подсистема прерываний в микроконтроллерах AVR реализована на основе векторной таблицы прерываний и включает источник прерывания, флаг прерывания и разрешающий бит. При возникновении события устанавливается флаг, после чего при разрешённых прерываниях (установленном глобальном бите I в регистре SREG) выполнение основной программы приостанавливается и происходит переход к соответствующему обработчику прерывания. После выполнения обработчика управление возвращается в основную программу с помощью инструкции RETI.

3. Как программно разрешить или запретить прерывание?

Программное разрешение или запрет прерываний осуществляется на двух уровнях: глобальном и локальном. Глобальное управление выполняется с помощью инструкций SEI (разрешить прерывания) и CLI (запретить прерывания), которые управляют битом I в регистре состояния SREG. Разрешение или запрет конкретного прерывания осуществляется установкой или сбросом соответствующего бита в регистрах управления прерываниями, например в регистре GICR для внешних прерываний.

4. Какие источники прерываний есть в AVR?

В микроконтроллерах AVR существует несколько источников прерываний. К ним относятся внешние прерывания (INT0, INT1), прерывания от таймеров и счётчиков (переполнение, совпадение), прерывания от интерфейсов связи (USART, SPI, TWI), от аналого-цифрового преобразователя (ADC), от EEPROM, от аналогового компаратора, а также прерывания сторожевого таймера (Watchdog). Это позволяет эффективно обрабатывать различные события без постоянного опроса.

5. Как настраиваются внешние прерывания?

Настройка внешних прерываний в AVR включает несколько этапов. Сначала задаётся условие срабатывания прерывания (по уровню, по фронту или по спаду сигнала) с помощью регистра MCUCR. Затем разрешается конкретное прерывание установкой соответствующего бита в регистре GICR. После этого при необходимости сбрасываются флаги прерываний в регистре GIFR. В завершение выполняется глобальное разрешение прерываний с помощью инструкции SEI.

ВЫВОД

В ходе лабораторной работы был изучен принцип работы портов ввода-вывода и системы прерываний микроконтроллера AVR. Реализована программа с несколькими режимами работы, переключаемыми с помощью внешних прерываний. Освоены настройка портов, обработка сигналов с кнопок, работа с EEPROM и формирование временных задержек. Получены практические навыки разработки и отладки программ для микроконтроллеров.

Листинг «.asm файл»

```
.include "m32def.inc"  ; Подключение файла с определениями для ATmega32 (имена регистров, битов и т.д.)

.equ EE_MODE = 0x00 ; Адрес в EEPROM для хранения MODE

.org 0x0000         ; Следующую команду размещаем по этому адресу
    rjmp RESET      ; Относительный переход на метку reset

.org INT0addr       ; Обработчик внешнего прерывания INT0
    rjmp EXT_INT0

.org INT1addr       ; Обработчик внешнего прерывания INT1
    rjmp EXT_INT1

; r17 - STATE
; r18 - MODE
; r19 - y



RESET:              ; Инициализация
    ; Инициализация стека
    ldi r16, high(RAMEND) ; ldi - загрузить константу в регистр
    out SPH, r16          ; загружаем старший байт адреса конца стека ( STACK POINTER HIGH = SPH )
    ldi r16, low(RAMEND)
    out SPL, r16          ; загружаем младший байт адреса конца стека ( STACK POINTER LOW = SPL )

    ; прочитать MODE из EEPROM
    ldi r16, EE_MODE    ; ldi - загрузка адреса 0x00, в к-ом хранится константа MODE

    ; Адрес EEPROM состоит из 10 бит: EEAR = [EEARH | EEARL]
```

```
    ; EEARL - младшие 8 бит (0–7) Address Register
    ; EEARH - старшие биты (8–9) Address Register

    out EEARL, r16      ; OUT - запись значения регистра в порт
    sbi EECR, EERE      ; SBI - установить бит в порту ввода-
вывода, EEPROM Read Enable Signal – EERE (если 1, прочитать из
EEPROM)
    ; после этого происходит чтение EEPROM[EEAR] -> EEDR
    in r18, EEDR        ; EEDR - регистр данных EEPROM (буфер
для данных)

    ; Проверка на соответствующее значение MODE
    cpi r18, 1          ; Сравнение регистр с константой Rd-k
    brlo MODE_FIX       ; Переход если меньше (C=1)
    cpi r18, 4
    brsh MODE_FIX       ; Переход если больше или равно (C=0)

    rjmp MODE_OK
    MODE_FIX:
       ldi r18, 1       ; Самый первый режим работы 0xFF <->
0x00
    MODE_OK:

    ; PORTA = output
    ldi r16, 0xFF       ; r16 = 1111 1111
    out DDRA, r16 ;  DDRA - регистр направления порта,
настраиваем на вывод
    out DDRB, r16 ;  DDRB - регистр направления порта,
настраиваем на вывод

    ; PORTD:
    ; PD0, PD1 - на выход, режим работы (00 - 1, 01 - 2, 11 - 3)
    ; PD4 - на выход, состояние STATE (1 <-> 0)
    ; PD2 - INT0 (вперёд), PD3 - INT1 (назад), PD7 (остановка
работы) = вход
    ; Если PD7 = 1 - читаем у с PORTC
```

```
    ldi r16, 0x13 ; 00010011
    out DDRD, r16 ; Устанавливаем PD0, PD1, PD4 на вывод,
остальное на ввод

    ; PORTC = вход
    clr r16
    out DDRC, r16 ; PORTC полностью на вход, т.к с него
считываться нажатия для y


    ; STATE = 0
    clr r17


    ; y = 0x55
    ldi r19, 0x55 ; r19 = 10101010

    rcall UPDATE_PORTD ; Вывести на PORTD текущий MODE и STATE

    ; INT0 и INT1 по спаду 0->1
    ; ISC01=1, ISC00=1
    ; ISC11=1, ISC10=1

    ldi r16, 0x0F  ; ----1111
    out MCUCR, r16 ; MCUCR – выбор условия срабатывания
прерываний int0 и int1


    ; разрешить INT0 и INT1
    ldi r16, (1<<INT0) | (1<<INT1)
    out GICR, r16 ; GICR – запрет/разрешение прерываний по
сигналам на входах INT0, INT1

    ; сбросить флаги INT0/INT1
    out GIFR, r16 ; GIFR – управление внешними прерываниями
```

```
    ; INTFx=1: произошло прерывание на входе INTx. При входе в
    ; подпрограмму обработки прерывания INTFx автоматически
сбрасывается
    ; в состояние логического нуля.

    sei ; Глобальное разрешение прерываний (Global interrupt)



MAIN:
    ; Если PD7 = 1 > режим ввода y
    sbis PIND, PD7      ; Пропустить след. команду, если бит = 1
    rjmp MAIN_CONT      ; Если 0 - обычный режим

    rcall READ_Y        ; Читаем y
MAIN_CONT:

    rcall SHOW_MODE
    rcall DELAY_250MS

    ldi r16, 1
    eor r17, r16        ; Меняем STATE
    rcall UPDATE_PORTD  ; Обновляем PD4

    rjmp MAIN


; =======================================
; SAVE_MODE: записать r18 в EEPROM
; =======================================
SAVE_MODE:
WAIT_EE:
    sbic EECR, EEWE ; Пропуск следующей команды если бит в порту
очищен

    ; EEWE = EEPROM Write Enable
```

```
    ; EEWE = 1 - идёт запись в EEPROM
    ; EEWE = 0 - EEPROM свободен

    rjmp WAIT_EE

    ldi r16, EE_MODE ; ldi - загрузка адреса 0x00, в к-ом
хранится константа MODE
    out EEARL, r16   ; Задаём адрес в EEPROM, куда запишем MODE

    out EEDR, r18    ; Записываем в регистр данных EEDR наш MODE
(01,10,11)

    sbi EECR, EEMWE  ; sbi - Установить бит в порту
ввода/вывода, разрешили запись
    sbi EECR, EEWE   ; Начали запись

    ret

; ========================================
; READ_Y: считывает y с PORTC при нажатой PD7
; ========================================
READ_Y:
READ_LOOP:
    in r19, PINC        ; читаем кнопки в y

    sbis PIND, PD7      ; Если бит = 1, пропускаем следующую
команду
    rjmp READ_DONE      ; Если отпустили PD7 > выходим

    rjmp READ_LOOP      ; Иначе крутимся в цикле

READ_DONE:
    ret

SHOW_MODE:
; Определяем MODE
```

```
    cpi r18, 1
    breq MODE1

    cpi r18, 2
    breq MODE2

    cpi r18, 3
    breq MODE3

    ret

; ================================ ;
; MODE 1: FF / 00
; ================================ ;
MODE1:
    tst r17 ; Определяем STATE
    breq MODE1_STATE0

MODE1_STATE1:
    ; состояние 1 -> 0x00
    ldi r16, 0x00
    out PORTA, r16
    out PORTB, r16
    ret

MODE1_STATE0:
    ; состояние 0 -> 0xFF
    ldi r16, 0xFF
    out PORTA, r16
    out PORTB, r16
    ret

; ================================ ;
; MODE 2: AA / 55
; ================================ ;
```

```
MODE2:
    tst r17 ; Определяем STATE
    breq MODE2_STATE0

MODE2_STATE1:
    ; состояние 1 -> 0x00
    ldi r16, 0x55
    out PORTA, r16
    out PORTB, r16
    ret

MODE2_STATE0:
    ; состояние 0 -> 0xFF
    ldi r16, 0xAA
    out PORTA, r16
    out PORTB, r16
    ret

; ================================ ;
; MODE 3: y / -y (доп. код)
; ================================ ;

MODE3:
    tst r17 ; Определяем STATE
    breq MODE3_STATE0

MODE3_STATE1:
    mov r16, r19
    neg r16        ; преобразование в доп. код
    out PORTA, r16
    out PORTB, r16
    ret

MODE3_STATE0:
    mov r16, r19
    out PORTA, r16
```

```
    out PORTB, r16
    ret


; ---------------------------------
; INT0: режим вперёд
; 1 -> 2 -> 3 -> 1
; ---------------------------------
EXT_INT0:
    in r16, SREG ; Считать регистр состояния процессора (SREG)
    push r16     ; Сохраняем в стеке

    inc r18      ; Переход к следующему MODE
    cpi r18, 4   ; Сравнение с константой Rd-k
    brne INT0_OK ; Переход если неравно (Z=0)
    ldi r18, 1

INT0_OK:
    rcall UPDATE_PORTD ; Обновляем вывод на PD0, PD1, PD4
    rcall SAVE_MODE    ; Сохраняем MODE в EEPROM

    pop r16
    out SREG, r16      ; Восстанавливаем состояние процессора
(SREG)
    reti

; ---------------------------------
; INT1: режим назад
; 1 <- 2 <- 3 <- 1
; ---------------------------------
EXT_INT1:
    in r16, SREG ; Считать регистр состояния процессора (SREG)
    push r16     ; Сохраняем в стеке

    cpi r18, 1    ; Сравнение с константой Rd-k
    brne INT1_DEC ; Переход если неравно (Z=0)
```

```
    ldi r18, 3
    rjmp INT1_OK

INT1_DEC:
    dec r18

INT1_OK:
    rcall UPDATE_PORTD ; Обновляем вывод на PD0, PD1, PD4
    rcall SAVE_MODE    ; Сохраняем MODE в EEPROM
    pop r16
    out SREG, r16      ; Восстанавливаем состояние процессора
(SREG)
    reti


; ========================================
; UPDATE_PORTD
; PD0-PD1 = номер режима
; PD4     = состояние
; r17 = STATE
; r18 = MODE
; ========================================
UPDATE_PORTD:
    in r20, PORTD           ; Считывание значения порта в
регистр
    andi r20, 0b11101100    ; andi - логическое И с константой.
Очистить PD0, PD1, PD4

    ; MODE -> PD0, PD1
    cpi r18, 1              ; Сравниваем с константой 1
    breq UPD_MODE1          ; Переход если равно (Z=1)

    cpi r18, 2              ; Сравниваем с константой 2
    breq UPD_MODE2          ; Переход если равно (Z=1)

    ; MODE = 3
```

```
    ori r20, 0b00000011      ; ori - Логическое ИЛИ с константой
    rjmp UPD_STATE

UPD_MODE1:
    ori r20, 0b00000001      ; Логическое ИЛИ с константой
    rjmp UPD_STATE

UPD_MODE2:
    ori r20, 0b00000010      ; Логическое ИЛИ с константой

UPD_STATE:
    tst r17                  ; Проверка r17 на 0
    breq UPD_OUT             ; Если 0 - STATE 0
    ori r20, 0b00010000      ; STATE = 1

UPD_OUT:
    out PORTD, r20           ; Формируем итоговый PORTD с нужным
MODE, STATE
    ret

; =========================
; delay ~31 ms
; =========================

delay_31ms:
    ldi r30, 244
    ldi r29, 147

delay_sub:
    inc r29
    nop
    brne delay_sub
    nop
    dec r30
    brne delay_sub
    ret
```

```
; =========================
; delay 250 ms
; =========================

DELAY_250MS:
    ldi r28, 8
d250_loop:
    rcall delay_31ms
    dec r28
    brne d250_loop
    ret
```